Feature Selection & Classification

Analyst: Joel Vinas

- Video Link: https://www.youtube.com/watch?v=6dLdKMOHgyA
- GitHub: https://github.com/joelvinas/COMP-SCI_5565/tree/main/Assignment%203
- Panopto Video: https://umsystem.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=2e1a6dd4-312b-479a-82ad-b3a9008a2127&start=0

Section 1: Cross-Validation & Bootstrapping

Google Colab Link:
https://colab.research.google.com/drive/1kSFziLj47zvwY1W8v2FTTCEDzbHG42IW?usp=sharing

```
1 install.packages("ISLR2")
```

```
Installing package into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)
```

```
1 #Coded by Joel Vinas
2 library(ISLR2)
3 library(scales)
4
5 useSeed <- 2148
6 AllElements <- 392
7 SizeOfSample <- AllElements / 2
8
9 set.seed(useSeed)
10 train <- sample(AllElements, SizeOfSample)
```

```r
print("1) Report the test rates for a Poly fit quadratic and cubic using the seed")
#Estimated test MSE for:

#Linear regression fit
lm.fit <- lm(mpg ~ horsepower, data = Auto, subset = train)
linearVal <- mean((Auto$mpg - predict(lm.fit, Auto))[-train]^2)
print(sprintf("Linear: %s", linearVal))

#Quadratic
lm.fit2 <- lm(mpg ~ poly(horsepower, 2), data = Auto,
    subset = train)
quadraticVal <- mean((mpg - predict(lm.fit2, Auto))[-train]^2)
print(sprintf("Quadratic: %s", quadraticVal))

#Cubic
lm.fit3 <- lm(mpg ~ poly(horsepower, 3), data = Auto,
    subset = train)
cubicVal <- mean((mpg - predict(lm.fit3, Auto))[-train]^2)
print(sprintf("Cubic: %s",cubicVal))
```

```
[1] "1) Report the test rates for a Poly fit quadratic and cubic using the seed"
[1] "Linear: 24.5122865552383"
[1] "Quadratic: 19.6288129137431"
[1] "Cubic: 19.6068646239259"
```

```r
useSeed <- 2148
set.seed(useSeed)
AllElements <- 392

ratioList <- c(5, 7, 9)

print("2) Compute and report the test performance for 3 new ratios")
for (curRatio in ratioList) {
  curRatioPct <- percent(1/curRatio, accuracy = 1)
  SizeOfSample <- AllElements / curRatio
  train <- sample(AllElements, SizeOfSample)
  print("========")
  print(sprintf("Ratio: %s (1/%s)", curRatioPct ,curRatio))

  #Estimated test MSE for:
  #Linear regression fit
  lm.fit <- lm(mpg ~ horsepower, data = Auto, subset = train)
  linearVal <- mean((Auto$mpg - predict(lm.fit, Auto))[-train]^2)
  print(sprintf("Linear: %s", linearVal))

  #Quadratic
  lm.fit2 <- lm(mpg ~ poly(horsepower, 2), data = Auto,
      subset = train)
  quadraticVal <- mean((mpg - predict(lm.fit2, Auto))[-train]^2)
  print(sprintf("Quadratic: %s", quadraticVal))

  #Cubic
  lm.fit3 <- lm(mpg ~ poly(horsepower, 3), data = Auto,
      subset = train)
  cubicVal <- mean((mpg - predict(lm.fit3, Auto))[-train]^2)
  print(sprintf("Cubic: %s",cubicVal))
  print("========")
}
```

```
[1] "2) Compute and report the test performance for 3 new ratios"
[1] "========"
[1] "Ratio: 20% (1/5)"
[1] "Linear: 24.0262637278904"
[1] "Quadratic: 19.7820945866969"
[1] "Cubic: 20.1040426564537"
[1] "========"
[1] "========"
[1] "Ratio: 14% (1/7)"
[1] "Linear: 24.1993312561485"
[1] "Quadratic: 19.088303354168"
[1] "Cubic: 19.886371524784"
[1] "========"
[1] "========"
[1] "Ratio: 11% (1/9)"
[1] "Linear: 24.5122865552383"
[1] "Quadratic: 19.6288129137431"
[1] "Cubic: 19.6068646239259"
[1] "========"
```

```r
print("3) Compute and report the overall performance of Poly for orders 1:8 for the weight feature in the Auto dataset to mpg.")
library(boot)

cv.error <- rep(0, 8)
for (i in 1:8) {
  glm.fit <- glm(mpg ~ poly(weight, i), data = Auto)
  cv.error[i] <- cv.glm(Auto, glm.fit)$delta[1]
}
cv.error
```

18.8516113114014 · 17.5247407933528 · 17.5781131048341 · 17.6232406315417 · 17.6282183319516 · 17.6941762821639 · 17.6669522963132 · 17.7645566755657

```r
print("Using acceleration, perform both a 5 fold and 10 fold k-fold cross validation.")
useSeed <- 2148
set.seed(useSeed)
totalPoly <- 8
kFolds <- 5

print("===  5-fold cross validation ===")
cv.error.totalPoly <- rep(0, totalPoly) #Initialize the vector
for (i in 1:totalPoly) {
  glm.fit <- glm(mpg ~ poly(acceleration, i), data = Auto)
  cv.error.totalPoly[i] <- cv.glm(Auto, glm.fit, K = kFolds)$delta[1]
}
cv.error.totalPoly

kFolds <- 10
print("=== 10-fold cross validation ===")
cv.error.totalPoly <- rep(0, totalPoly) #Initialize the vector
for (i in 1:totalPoly) {
  glm.fit <- glm(mpg ~ poly(acceleration, i), data = Auto)
  cv.error.totalPoly[i] <- cv.glm(Auto, glm.fit, K = kFolds)$delta[1]
}
cv.error.totalPoly
```

```
[1] "Using acceleration, perform both a 5 fold and 10 fold k-fold cross validation."
[1] "===  5-fold cross validation ==="
50.217054227902 · 49.3192255823285 · 50.3272253843509 · 50.4110381850219 · 49.8109171299611 · 50.260524557834 · 51.1863798305747 · 51.3051835260855
[1] "=== 10-fold cross validation ==="
50.3042822438261 · 49.6306303389681 · 50.0635741993196 · 49.4811668916228 · 48.9774151816722 · 49.9626379361924 · 50.9844607356672 · 51.8455487033712
```

5. Compute the last bootstrap exercise (the quadratic fit for horsepower) from the lab 3 more times, using a new number of samples {250, 500, 2500}. The goal of this task will be to compare the error estimates of the increasing number of samples. Report your observations about the error as the number of bootstrap sets increases.

```r
#References for further reading: https://stats.oarc.ucla.edu/r/faq/how-can-i-generate-bootstrap-statistics-in-r/

print("5) Compute the quadratic fit for horsepower bootstrap using the a new number of samples {250, 500, 2500}.")
#The goal of this task will be to compare the error estimates of the increasing number of samples.
#Report your observations about the error as the number of bootstrap sets increases.
useSeed <- 2148


boot.fn <- function(data, index)
  coef(lm(mpg ~ horsepower + I(horsepower^2),data = data, subset = index))
set.seed(2148)


# bootResult <- boot(Auto, boot.fn, 250)
# bootResult
#summary(bootResult)
#plot(bootResult)

#bootResult <- boot(Auto, boot.fn, 500)
#plot(bootResult)

#bootResult <- boot(Auto, boot.fn, 2500)
#plot(bootResult)

#print(bootResult$t)
#print(mean(bootResult$t) - bootResult$t0)

sampleList <- c(250,500, 2500)
for (curSample in sampleList) {
  print(sprintf("====Number of samples: %s====", curSample))
  bootResult <- boot(Auto, boot.fn, curSample)
  t1bias <-mean(bootResult$t[,1]) - bootResult$t0[1]
  t2bias <-mean(bootResult$t[,2]) - bootResult$t0[2]
  t3bias <-mean(bootResult$t[,3]) - bootResult$t0[3]
  print("====Bias====")
  cat("Intercept", t1bias,"\n")
  cat("Horsepower", t2bias,"\n")
  cat("Horsepower^2", t3bias,"\n")

  t1Stde <- sd(bootResult$t[,1])
  t2Stde <- sd(bootResult$t[,2])
  t3Stde <- sd(bootResult$t[,3])
  print("====Standard Error====")
  cat("Intercept",t1Stde, "\n")
  cat("Horsepower",t2Stde, "\n")
  cat("Horsepower^2",t3Stde, "\n")
  print("================================")
}


#print(sd(bootResult$t))
#print(sd(bootResult$t0))

#sampleList <- c(250,500, 2500)
#for (curSample in sampleList) {
#  print(sprintf("====Number of samples: %s====", curSample))
#  bootResult <- boot(Auto, boot.fn, curSample)
  #print(boot(Auto, boot.fn, curSample))
  #plot(bootResult)
  #print(bootResult, type="bias")
#  print("================================")
#}
print(summary(lm(mpg ~ horsepower + I(horsepower^2), data = Auto))$coef)

print("====Analysis=====")
print("As the number of samples increases from 250 to 500, the error decreases. It then increases as the number of samples grows.")
print("Noting that the original sample size was 392 vehicles, this makes sense as the results force artificial standard distribution.")
```

```
[1] "5) Compute the quadratic fit for horsepower bootstrap using the a new number of samples {250, 500, 2500}."
[1] "====Number of samples: 250===="
[1] "====Bias===="
Intercept -0.07146366
Horsepower 0.0004867345
Horsepower^2 -7.769899e-07
[1] "====Standard Error===="
Intercept 2.111568
Horsepower 0.03326454
Horsepower^2 0.0001195684
[1] "================================="
[1] "====Number of samples: 500===="
[1] "====Bias===="
Intercept -0.01812876
Horsepower 0.0002455549
Horsepower^2 -9.243604e-07
[1] "====Standard Error===="
Intercept 2.005874
Horsepower 0.03203003
Horsepower^2 0.0001159558
[1] "================================="
[1] "====Number of samples: 2500===="
[1] "====Bias===="
Intercept 0.01183895
Horsepower -0.0004921367
Horsepower^2 2.664252e-06
[1] "====Standard Error===="
Intercept 2.109284
Horsepower 0.03375427
Horsepower^2 0.000122129
[1] "================================="
                    Estimate   Std. Error   t value      Pr(>|t|)
(Intercept)     56.900099702 1.8004268063  31.60367 1.740911e-109
horsepower      -0.466189630 0.0311246171 -14.97816  2.289429e-40
I(horsepower^2)  0.001230536 0.0001220759  10.08009  2.196340e-21
[1] "====Analysis====="
[1] "As the number of samples increases, the error decreases from 250 to 500, then increases as the size grows."
[1] "Noting that the original sample size was 392 vehicles, this makes sense as the results force artificial standard distribution."
```

Section 2: Classification

Google Colab Link:
https://colab.research.google.com/drive/1uXR2pTF0KrktCcWRqm9jGFCT0rhQK6LI?usp=sharing

Tasks

(1) Using the dataset provided above

- Use your plot functions to determine if the distributions are well suited for classification tasks!
- Start by implementing your own version of the "Logistic Regression" exercise above. --> Please provide the code you used to train your model, the summary of the model, and a plot of your logistic regression model. The following link includes 2 methods to generate this plot: (https://www.geeksforgeeks.org/how-to-plot-a-logistic-regression-curve-in-r/) Links to an external site.

(2) Using the same data from part (1),

- Apply the "Linear Discriminate" method. Include the class predictions for your trained model, then plot the results for the and compare the results. -- > Please provide your code, relevant summary/metrics and plots for your model using plot(lda.fit) [as given in the lab] or pairs.lda (MASS).

(3) Implement and assess a Classification model:

1. Select a dataset and classification TARGET and select the relevant features you wish to use. [You may use data from ISLR or MASS -- or one of the sets we've worked with so far]. You can either choose a binary classification (where applicable) or a multi-class target using the LDA method. You may use either a single factor classification (only one feature) or multiple features (I suggest you limit your selection to 5). -- No performance metrics are required in this case, we will assess feature selection in a later assignment. Select the division scale (test set size) and perform a fit for your model on the training data.
2. Repeat the process from the previous step using the k-fold methods: Recall selection of k-fold number is important, you may select any valid value for the range we've discussed.

Questions

1. Did your performance assessment of the model generated change based on the results of your k-fold analsys?
2. What properties or limitations of your dataset (or selected predictors) do you feel most contributed to shortcomings in model performance (if your models are very low accuracy, be sure to emphasize this response as it may help your score to be able to explain why your efforts didn't generate good* results). *good results are relative to each dataset, but true classification ratios of <60% are generally considered very limited in their utility.

```
[ ]
4s
1 install.packages("ISLR2")
```

```
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
```

```
#Import WDBC Data
#(1) Use the dataset provided above
source_file_url = "https://raw.githubusercontent.com/joelvinas/COMP-SCI_5565/refs/heads/main/Assignment%203/Data/wdbc.data"

wdbc_data <- read.csv(source_file_url)
all_headers <- c("ID", "Diagnosis",
                 "radius1", "texture1", "perimeter1", "area1", "smoothness1", "compactness1", "concavity1", "concave_points1", "symmetry1", "fractal_dimension1",
                 "radius2", "texture2", "perimeter2", "area2", "smoothness2", "compactness2", "concavity2", "concave_points2", "symmetry2", "fractal_dimension2",
                 "radius3", "texture3", "perimeter3", "area3", "smoothness3", "compactness3", "concavity3", "concave_points3", "symmetry3", "fractal_dimension3")

colnames(wdbc_data) <- all_headers
#wdbc_data$Diagnosis[wdbc_data$Diagnosis == "M"] <- 1   #M for Malignant
#wdbc_data$Diagnosis[wdbc_data$Diagnosis == "B"] <- 0  #B for Benign

# Gemini Code suggestion: Robustly convert Diagnosis to numeric 0/1 (0 for 'B', 1 for 'M')
wdbc_data$Diagnosis <- as.numeric(wdbc_data$Diagnosis == "M")

#Break the dataset into 5 parts for later 5-Fold analysis
num_parts <- 5
wdbc_data$DataGroup <- rep(1:num_parts, length.out = nrow(wdbc_data))

head(wdbc_data)

#Split the dataset into the 3 dimensions
wdbc_data1 <- wdbc_data[, c(1,2, 3:12, 33)]
wdbc_data2 <- wdbc_data[, c(1,2, 13:22, 33)]
wdbc_data3 <- wdbc_data[, c(1,2, 23:32, 33)]

# Define new headers for subset
part_headers <- c("ID", "Diagnosis", "radius", "texture", "perimeter", "area", "smoothness", "compactness", "concavity", "concave_points", "symmetry", "fractal_dimension", "data_group")

# Assign the new headers to the new data frames
colnames(wdbc_data1) <- part_headers
colnames(wdbc_data2) <- part_headers
colnames(wdbc_data3) <- part_headers

head(wdbc_data1)
head(wdbc_data2)
head(wdbc_data3)
```

A data.frame: 6 × 33

	ID	Diagnosis	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	...	texture3	perimeter3	area3	smoothness3	compactness3	concavity3	concave_points3	symmetry3	fractal_dimension3	DataGroup
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	...	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
1	842517	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	1
2	84300903	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	2
3	84348301	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	3
4	84358402	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	4
5	843786	1	12.45	15.70	82.57	477.1	0.12780	0.17000	0.1578	0.08089	...	23.75	103.40	741.6	0.1791	0.5249	0.5355	0.1741	0.3985	0.12440	5
6	844359	1	18.25	19.98	119.60	1040.0	0.09463	0.10900	0.1127	0.07400	...	27.66	153.20	1606.0	0.1442	0.2576	0.3784	0.1932	0.3063	0.08368	1

A data.frame: 6 × 13

	ID	Diagnosis	radius	texture	perimeter	area	smoothness	compactness	concavity	concave_points	symmetry	fractal_dimension	data_group
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
1	842517	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	1
2	84300903	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	2
3	84348301	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	3
4	84358402	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	4
5	843786	1	12.45	15.70	82.57	477.1	0.12780	0.17000	0.1578	0.08089	0.2087	0.07613	5
6	844359	1	18.25	19.98	119.60	1040.0	0.09463	0.10900	0.1127	0.07400	0.1794	0.05742	1

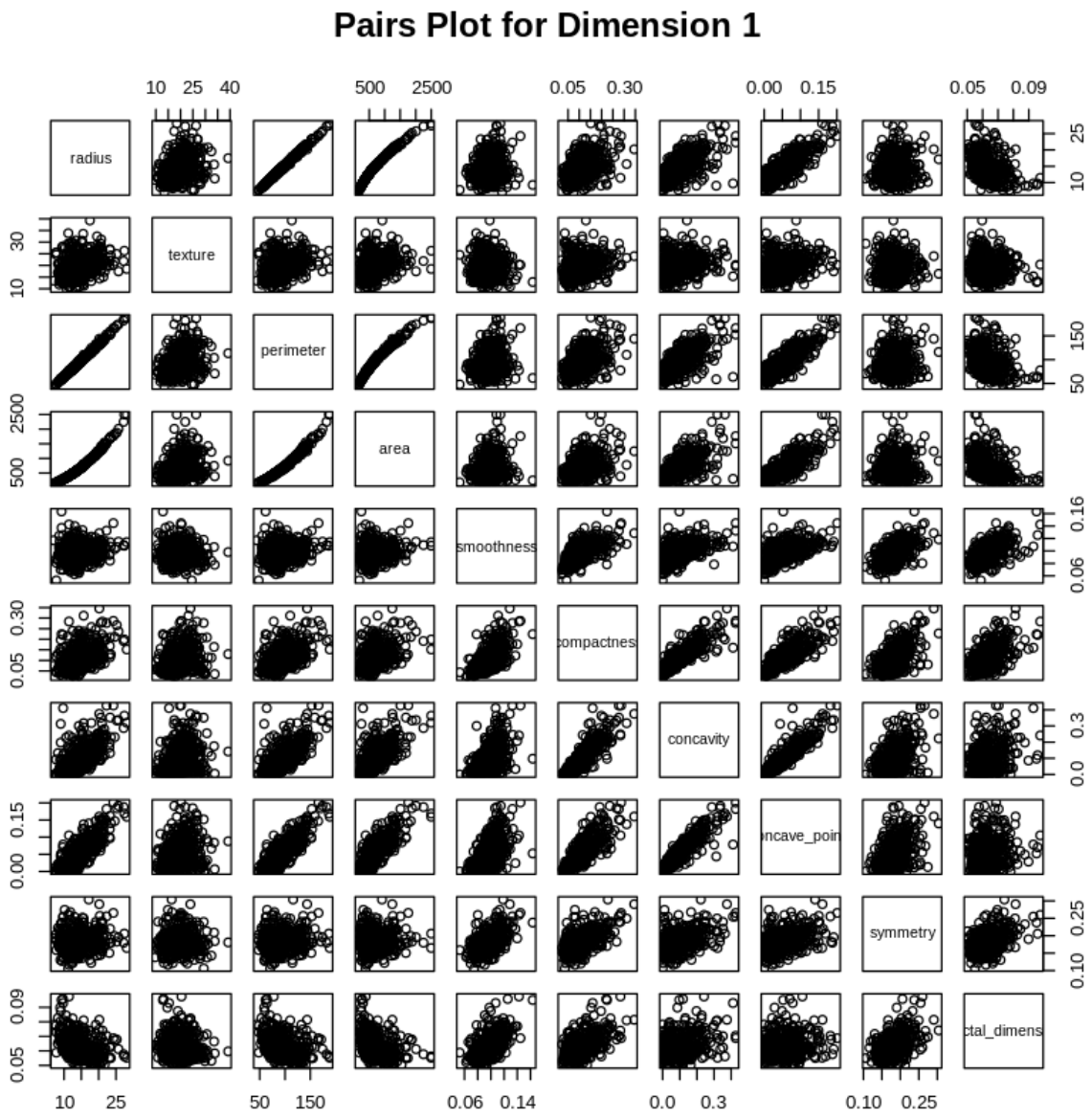
A data.frame: 6 × 13

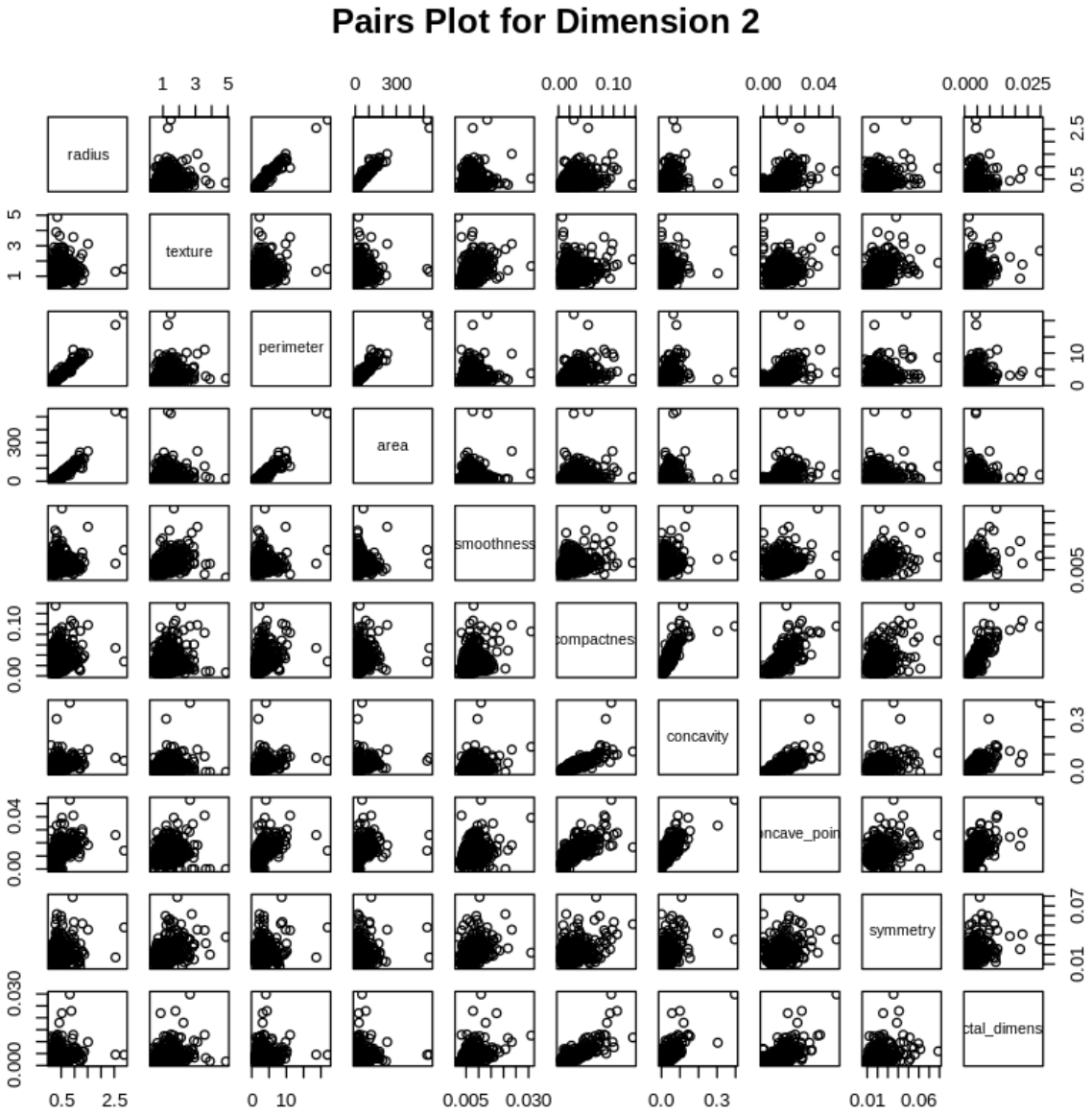
	ID	Diagnosis	radius	texture	perimeter	area	smoothness	compactness	concavity	concave_points	symmetry	fractal_dimension	data_group
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
1	842517	1	0.5435	0.7339	3.398	74.08	0.005225	0.01308	0.01860	0.01340	0.01389	0.003532	1
2	84300903	1	0.7456	0.7869	4.585	94.03	0.006150	0.04006	0.03832	0.02058	0.02250	0.004571	2
3	84348301	1	0.4956	1.1560	3.445	27.23	0.009110	0.07458	0.05661	0.01867	0.05963	0.009208	3
4	84358402	1	0.7572	0.7813	5.438	94.44	0.011490	0.02461	0.05688	0.01885	0.01756	0.005115	4
5	843786	1	0.3345	0.8902	2.217	27.19	0.007510	0.03345	0.03672	0.01137	0.02165	0.005082	5
6	844359	1	0.4467	0.7732	3.180	53.91	0.004314	0.01382	0.02254	0.01039	0.01369	0.002179	1

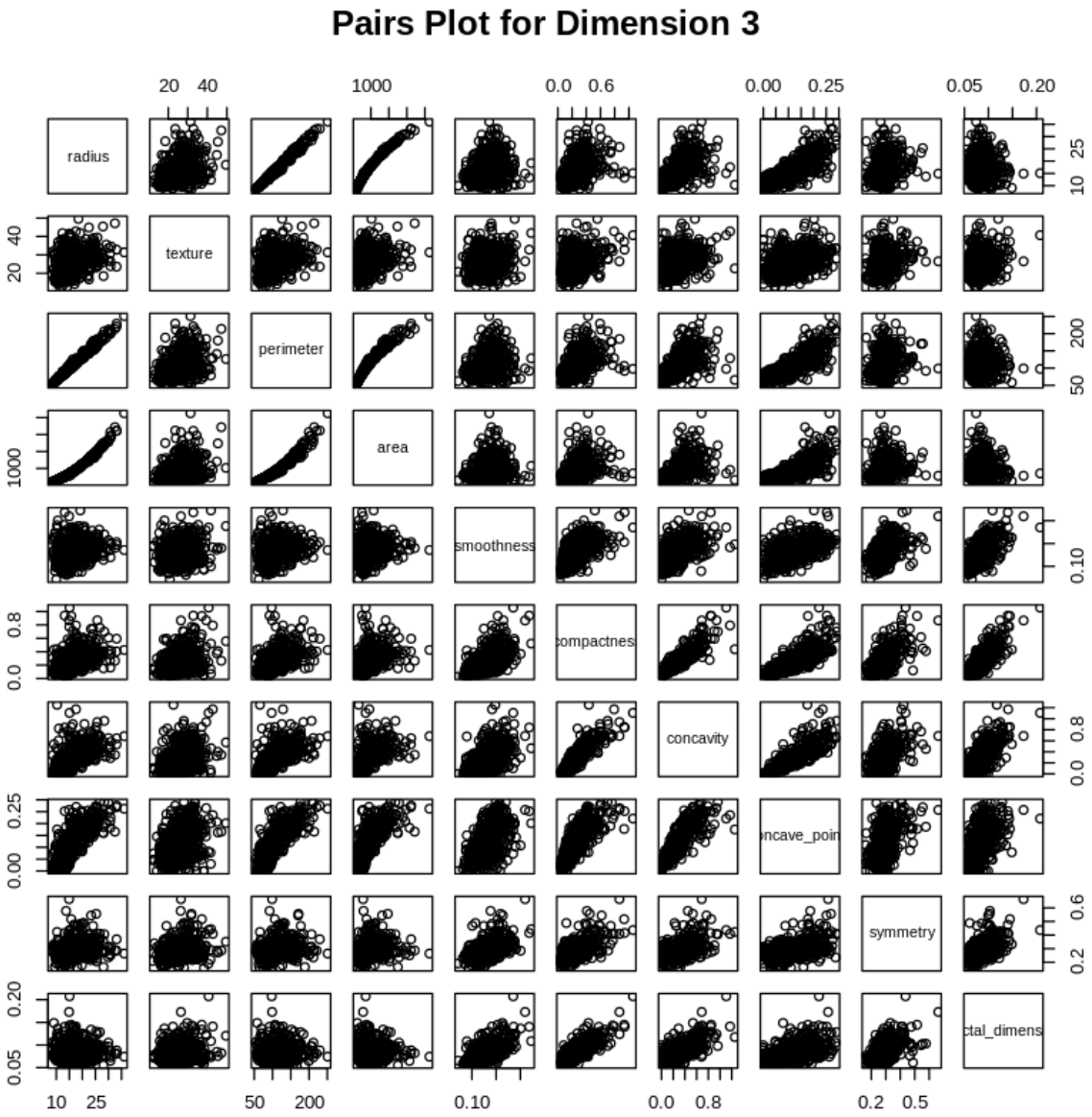
A data.frame: 6 × 13

	ID	Diagnosis	radius	texture	perimeter	area	smoothness	compactness	concavity	concave_points	symmetry	fractal_dimension	data_group
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
1	842517	1	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	1
2	84300903	1	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	2
3	84348301	1	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	3
4	84358402	1	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	4
5	843786	1	15.47	23.75	103.40	741.6	0.1791	0.5249	0.5355	0.1741	0.3985	0.12440	5
6	844359	1	22.88	27.66	153.20	1606.0	0.1442	0.2576	0.3784	0.1932	0.3063	0.08368	1

```
#Use plot functions to determine if the distributions are well suited for classification tasks
# Set par(mfrow = c(1,1)) to display each plot individually for larger size
par(mfrow = c(1,1))
pairs(wdbc_data1[, -c(1:2,13)], main = 'Pairs Plot for Dimension 1')
pairs(wdbc_data2[, -c(1:2,13)], main = 'Pairs Plot for Dimension 2')
pairs(wdbc_data3[, -c(1:2,13)], main = 'Pairs Plot for Dimension 3')
par(mfrow = c(1,1)) # Reset plot layout after the last plot
```


Pairs Plot for Dimension 1
radius
texture
perimeter
area
smoothness
compactnes
concavity
ncave_poin
symmetry
ctal_dimens


Pairs Plot for Dimension 2
radius
texture
perimeter
area
smoothness
compactnes
concavity
ncave_poin
symmetry
ctal_dimens



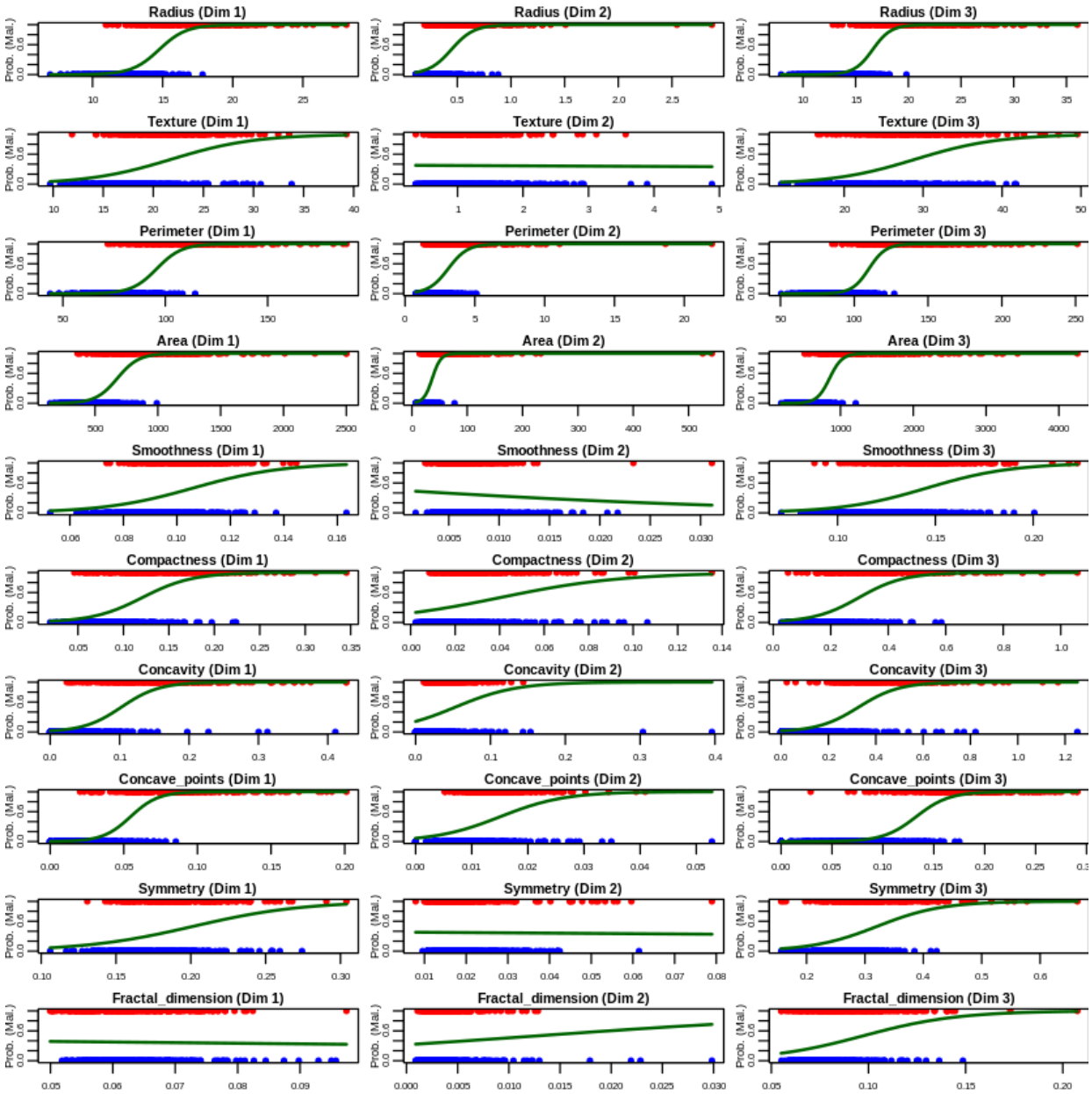

```r
#Start by implementing your own version of the "Logistic Regression" exercise above. -->
#Please provide the code you used to train your model, the summary of the model, and a plot of your logistic regression model.
#The following link includes 2 methods to generate this plot: (https://www.geeksforgeeks.org/how-to-plot-a-logistic-regression-curve-in-r/) Links to an external site.
#logistic_model1 <- glm(var1 ~ var2, data=wdbc_data1, family=binomial)
#predicted_data <- data.frame(var2=seq(min(), max(), len=500))

dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

for (df_name in names(dataframe_list)) {
  current_df <- dataframe_list[[df_name]]

  glm.fits <- suppressWarnings(
    glm(Diagnosis ~ radius + texture + perimeter + area + smoothness + compactness + concavity + concave_points + symmetry + fractal_dimension,
      data = current_df,
      family = binomial)
    )
  print(paste("====", df_name, "===="))
  print(summary(glm.fits))
}
```

```
[1] "==== Dim 1 ===="

Call:
glm(formula = Diagnosis ~ radius + texture + perimeter + area +
    smoothness + compactness + concavity + concave_points + symmetry +
    fractal_dimension, family = binomial, data = current_df)

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)       -7.35983   12.85270  -0.573   0.5669
radius            -2.04923    3.71590  -0.551   0.5813
texture            0.38473    0.06454   5.961  2.5e-09 ***
perimeter         -0.07151    0.50516  -0.142   0.8874
area               0.03980    0.01674   2.377   0.0174 *
smoothness        76.43231   31.95490   2.392   0.0168 *
compactness       -1.46251   20.34247  -0.072   0.9427
concavity          8.46881    8.12006   1.043   0.2970
concave_points    66.82118   28.52923   2.342   0.0192 *
symmetry          16.27816   10.63058   1.531   0.1257
fractal_dimension -68.33655   85.55655  -0.799   0.4244
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 749.46  on 567  degrees of freedom
Residual deviance: 146.13  on 557  degrees of freedom
AIC: 168.13

Number of Fisher Scoring iterations: 9
```

```
[1] "==== Dim 2 ===="

Call:
glm(formula = Diagnosis ~ radius + texture + perimeter + area +
    smoothness + compactness + concavity + concave_points + symmetry +
    fractal_dimension, family = binomial, data = current_df)

Coefficients:
                    Estimate Std. Error z value Pr(>|z|)
(Intercept)         -3.39225    0.63580  -5.335 9.53e-08 ***
radius             -28.22915    6.13789  -4.599 4.24e-06 ***
texture             -0.33386    0.42301  -0.789   0.4300
perimeter            0.07511    0.47621   0.158   0.8747
area                 0.39346    0.05710   6.891 5.56e-12 ***
smoothness           4.19959   98.69202   0.043   0.9661
compactness         57.25377   28.68816   1.996   0.0460 *
concavity            3.61415   11.95153   0.302   0.7623
concave_points      39.76686   64.17421   0.620   0.5355
symmetry            -6.76193   30.45760  -0.222   0.8243
fractal_dimension -318.73330  149.29380  -2.135   0.0328 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 749.46  on 567  degrees of freedom
Residual deviance: 265.92  on 557  degrees of freedom
AIC: 287.92

Number of Fisher Scoring iterations: 8
```

```
[1] "==== Dim 3 ===="

Call:
glm(formula = Diagnosis ~ radius + texture + perimeter + area +
    smoothness + compactness + concavity + concave_points + symmetry +
    fractal_dimension, family = binomial, data = current_df)

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)      -29.00148   13.49448  -2.149   0.0316 *
radius            -0.53543    1.53087  -0.350   0.7265
texture            0.28250    0.06004   4.705 2.54e-06 ***
perimeter          0.01300    0.12846   0.101   0.9194
area               0.01878    0.01465   1.282   0.1998
smoothness        53.94341   21.87952   2.465   0.0137 *
compactness       -8.31719    8.44721  -0.985   0.3248
concavity          4.57985    3.37161   1.358   0.1744
concave_points    37.54868   16.15536   2.324   0.0201 *
symmetry           9.62227    5.84695   1.646   0.0998 .
fractal_dimension -7.87460   49.29051  -0.160   0.8731
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 749.462  on 567  degrees of freedom
Residual deviance:  83.567  on 557  degrees of freedom
AIC: 105.57

Number of Fisher Scoring iterations: 9
```

```r
#!! The below code was produced with assistance from Google Gemini !!
#!! Prompt and response can be found within the AI_Prompts directory

#==Generalized Logistic Regression Plotting Function==
#This code block introduces a plot_logistic_regression function that takes a predictor variable name as an argument
#This allows logistic regression plots for different features without rewriting the code each time.

plot_logistic_regression <- function(predictor_name, data_frame, data_name) {
  # Create logistic regression model using the specified predictor
  formula_str <- paste0("Diagnosis ~ ", predictor_name)
  logistic_model <- suppressWarnings(glm(
    as.formula(formula_str),
    data = data_frame,
    family = binomial
  ))

  # Get the range for the chosen predictor
  predictor_range <- seq(min(data_frame[[predictor_name]]), max(data_frame[[predictor_name]]), len = 500)

  # Calculate the means of all predictor variables in data_frame (excluding 'ID' and 'Diagnosis')
  predictor_means <- colMeans(data_frame[, !(names(data_frame) %in% c("ID", "Diagnosis"))])

  # Create a data frame for predictions with the specified predictor varying and others at their means
  Predicted_data <- data.frame(matrix(ncol = length(names(predictor_means)), nrow = 500))
  colnames(Predicted_data) <- names(predictor_means)

  Predicted_data[[predictor_name]] <- predictor_range
  for (col_name in names(predictor_means)) {
    if (col_name != predictor_name) { # Set other predictors to their mean
      Predicted_data[[col_name]] <- predictor_means[col_name]
    }
  }

  # Fill predicted values using regression model
  Predicted_data$prob <- predict(logistic_model, Predicted_data, type = "response")

  # Plot Predicted data and original data points
  # Shorten plot title and y-axis label, and reduce font sizes for better fitting in multi-plot layout
  plot_title <- paste0(tools::toTitleCase(predictor_name), " (", data_name, ")")
  # xlab = tools::toTitleCase(predictor_name) - Replaced with null to make more visible
  plot(data_frame[[predictor_name]], data_frame$Diagnosis,
       xlab = "", ylab = "Prob. (Mal.)",
       main = plot_title,
       pch = 20, col = ifelse(data_frame$Diagnosis == 1, "red", "blue"),
       cex.axis = 0.6, cex.lab = 0.7, cex.main = 0.8
       ) # Differentiate points by Diagnosis
  lines(Predicted_data[[predictor_name]], Predicted_data$prob, col = "darkgreen", lwd = 2)
}

# Set global plot layout and significantly smaller margins/text sizes for all plots
par(mfrow = c(10, 3), mar = c(1.5, 1.5, 1, 0.5) + 0.1, oma = c(0, 0, 0, 0), mgp = c(1, 0.5, 0))

dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

# Get the list of all predictor variables (excluding ID and Diagnosis)
predictor_vars <- names(wdbc_data1)[!(names(wdbc_data1) %in% c("ID", "Diagnosis"))]

for (pred_var in predictor_vars) {

  # Loop through each predictor and generate a plot for the current dataframe
  for (df_name in names(dataframe_list)) {
    current_df <- dataframe_list[[df_name]]
    plot_logistic_regression(pred_var, current_df, df_name)
  }
}
# Reset plot layout and margins to default after all plots are generated
par(mfrow = c(1,1), mar = c(5, 4, 4, 2) + 0.1, oma = c(0, 0, 0, 0), mgp = c(3, 1, 0))
```


Radius (Dim 1)
Radius (Dim 2)
Radius (Dim 3)
Texture (Dim 1)
Texture (Dim 2)
Texture (Dim 3)
Perimeter (Dim 1)
Perimeter (Dim 2)
Perimeter (Dim 3)
Area (Dim 1)
Area (Dim 2)
Area (Dim 3)
Smoothness (Dim 1)
Smoothness (Dim 2)
Smoothness (Dim 3)
Compactness (Dim 1)
Compactness (Dim 2)
Compactness (Dim 3)
Concavity (Dim 1)
Concavity (Dim 2)
Concavity (Dim 3)
Concave_points (Dim 1)
Concave_points (Dim 2)
Concave_points (Dim 3)
Symmetry (Dim 1)
Symmetry (Dim 2)
Symmetry (Dim 3)
Fractal_dimension (Dim 1)
Fractal_dimension (Dim 2)
Fractal_dimension (Dim 3)
Prob. (Mal.)

```r
dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

for (df_name in names(dataframe_list)) {
  current_df <- dataframe_list[[df_name]]

  glm.fits <- suppressWarnings(
  glm(Diagnosis ~ radius + texture + perimeter + area + smoothness + compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = current_df,
    family = binomial)
  )
  print(paste("===",df_name,"==="))
  #summary(glm.fits)

  #print(coef(glm.fits))

  print(summary(glm.fits)$coef)

  glm.probs <- predict(glm.fits, type = "response")
  print(glm.probs[1:10])

  # Adjust length of glm.pred to match the number of rows in the dataset (569)
  glm.pred <- rep(0, nrow(current_df))
  glm.pred[glm.probs > .5] = 1
  print(table(glm.pred, current_df$Diagnosis))

  glm.probs <- predict(glm.fits, type = "response")
  print(glm.probs[1:10])

  print(paste(df_name, mean(glm.pred == current_df$Diagnosis)))
}


#summary(glm.fits)$coef[, 4]

```

```
[1] "=== Dim 1 ==="
                     Estimate  Std. Error    z value     Pr(>|z|)
(Intercept)        -7.35983192 12.85269862 -0.5726293 5.668957e-01
radius             -2.04922616  3.71589697 -0.5514755 5.813078e-01
texture             0.38473361  0.06453673  5.9614675 2.499827e-09
perimeter          -0.07151462  0.50516434 -0.1415670 8.874220e-01
area                0.03979558  0.01673994  2.3772831 1.744070e-02
smoothness         76.43230828 31.95489615  2.3918810 1.676228e-02
compactness        -1.46251366 20.34246884 -0.0718946 9.426858e-01
concavity           8.46881174  8.12005583  1.0429499 2.969715e-01
concave_points     66.82118162 28.52922646  2.3422010 1.917039e-02
symmetry           16.27815806 10.63058179  1.5312575 1.257058e-01
fractal_dimension -68.33654956 85.55654560 -0.7987296 4.244472e-01
        1         2         3         4         5         6         7         8
0.9999894 0.9999999 0.9822755 0.9999987 0.6692695 0.9995616 0.7736619 0.9923905
        9        10
0.9647319 0.6048361

glm.pred   0   1
       0 347  19
       1  10 192
        1         2         3         4         5         6         7         8
0.9999894 0.9999999 0.9822755 0.9999987 0.6692695 0.9995616 0.7736619 0.9923905
        9        10
0.9647319 0.6048361
[1] "Dim 1 0.948943661971831"
```

```
[1] "=== Dim 2 ==="
                       Estimate  Std. Error    z value     Pr(>|z|)
(Intercept)         -3.39224530   0.6357988 -5.3354067 9.533064e-08
radius             -28.22915077   6.1378867 -4.5991645 4.241887e-06
texture             -0.33386117   0.4230083 -0.7892543 4.299634e-01
perimeter            0.07510587   0.4762085  0.1577164 8.746803e-01
area                 0.39346311   0.0571016  6.8905799 5.556542e-12
smoothness           4.19959246  98.6920180  0.0425525 9.660583e-01
compactness         57.25376773  28.6881563  1.9957284 4.596350e-02
concavity            3.61414860  11.9515329  0.3024004 7.623468e-01
concave_points      39.76686093  64.1742082  0.6196705 5.354748e-01
symmetry            -6.76192703  30.4576036 -0.2220111 8.243052e-01
fractal_dimension -318.73330397 149.2937994 -2.1349400 3.276591e-02
          1           2           3           4           5           6
0.999974438 0.999999388 0.007549939 0.999998154 0.182129208 0.996688373
          7           8           9          10
0.652098878 0.211080017 0.254091742 0.809051193

glm.pred   0   1
       0 342  42
       1  15 169
          1           2           3           4           5           6
0.999974438 0.999999388 0.007549939 0.999998154 0.182129208 0.996688373
          7           8           9          10
0.652098878 0.211080017 0.254091742 0.809051193
[1] "Dim 2 0.899647887323944"
```

```
[1] "=== Dim 3 ==="
                      Estimate  Std. Error    z value     Pr(>|z|)
(Intercept)       -29.00147693 13.49448111 -2.1491361 3.162361e-02
radius             -0.53543197  1.53086855 -0.3497570 7.265211e-01
texture             0.28250077  0.06004175  4.7050722 2.537761e-06
perimeter           0.01299866  0.12846076  0.1011878 9.194014e-01
area                0.01877873  0.01464520  1.2822451 1.997567e-01
smoothness         53.94340511 21.87952361  2.4654744 1.368320e-02
compactness        -8.31719132  8.44720547 -0.9846086 3.248164e-01
concavity           4.57985115  3.37160953  1.3583575 1.743503e-01
concave_points     37.54868057 16.15536361  2.3242238 2.011350e-02
symmetry            9.62227059  5.84694985  1.6456906 9.982747e-02
fractal_dimension  -7.87459920 49.29050735 -0.1597589 8.730710e-01
        1         2         3         4         5         6         7         8
1.0000000 1.0000000 0.9878614 0.9999726 0.8623960 0.9999998 0.9605935 0.9927249
        9        10
0.9978958 0.9943192

glm.pred   0   1
       0 353   7
       1   4 204
        1         2         3         4         5         6         7         8
1.0000000 1.0000000 0.9878614 0.9999726 0.8623960 0.9999998 0.9605935 0.9927249
        9        10
0.9978958 0.9943192
[1] "Dim 3 0.980633802816901"
```

```r
library(dplyr)
#set.seed(2148)

dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

for (df_name in names(dataframe_list)) {
  current_df <- dataframe_list[[df_name]]

  for (cur_group in 1:5) {
  trainData <- current_df %>%
    filter(data_group != cur_group)
  testData <- current_df %>%
    filter(data_group == cur_group)

  glm.fits <- suppressWarnings(
    glm(Diagnosis ~ radius + texture + perimeter + area + smoothness + compactness + concavity + concave_points + symmetry + fractal_dimension,
      data = trainData,
      family = binomial)
    )

  glm.probs <- predict(glm.fits, testData, type = "response")

  totalRows <- nrow(testData)

  glm.pred <- rep(0, totalRows)
  glm.pred[glm.probs > 0.5] <- 1

  print(paste(df_name,": ", cur_group))
  print(table(glm.pred, testData$Diagnosis))
  print(paste("Train:",mean(glm.pred != testData)))
  print(paste("Test: ",mean(glm.pred == testData)))
  }
}
```

```
[1] "Dim 1 :  1"

glm.pred  0  1
       0 74  5
       1  2 33
[1] "Train: 0.900134952766532"
[1] "Test:  0.0998650472334683"
[1] "Dim 1 :  2"

glm.pred  0  1
       0 60  1
       1  4 49
[1] "Train: 0.923751686909582"
[1] "Test:  0.0762483130904184"
[1] "Dim 1 :  3"

glm.pred  0  1
       0 67  2
       1  5 40
[1] "Train: 0.923751686909582"
[1] "Test:  0.0762483130904184"
[1] "Dim 1 :  4"

glm.pred  0  1
       0 70  7
       1  1 35
[1] "Train: 0.925799863852961"
[1] "Test:  0.0742001361470388"
[1] "Dim 1 :  5"

glm.pred  0  1
       0 71  5
       1  3 34
[1] "Train: 0.924438393464942"
[1] "Test:  0.0755616065350579"
```

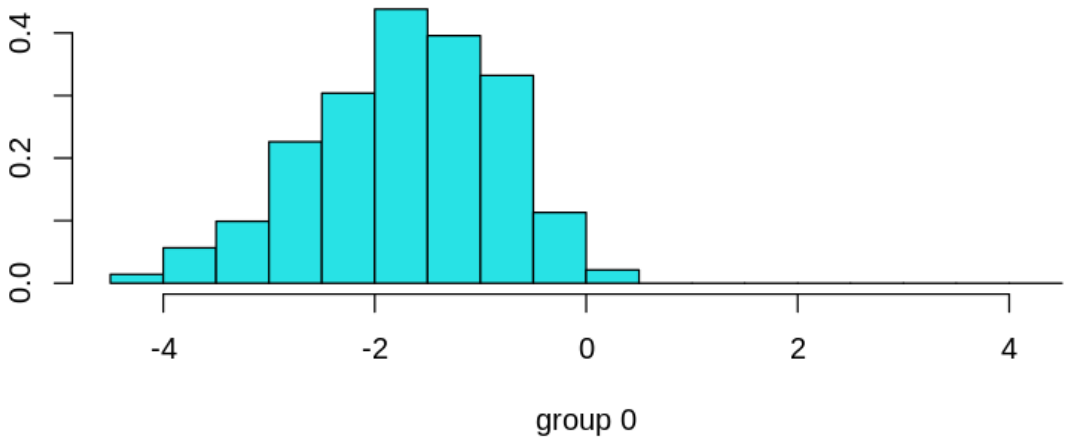
```
[1] "Dim 2 :  1"

glm.pred  0  1
       0 71  9
       1  5 29
[1] "Train: 0.904858299595142"
[1] "Test:  0.0951417004048583"
[1] "Dim 2 :  2"

glm.pred  0  1
       0 62  4
       1  2 46
[1] "Train: 0.92442645074224"
[1] "Test:  0.0755735492577598"
[1] "Dim 2 :  3"

glm.pred  0  1
       0 63  7
       1  9 35
[1] "Train: 0.929824561403509"
[1] "Test:  0.0701754385964912"
[1] "Dim 2 :  4"

glm.pred  0  1
       0 70 14
       1  1 28
[1] "Train: 0.930565010211028"
[1] "Test:  0.0694349897889721"
[1] "Dim 2 :  5"

glm.pred  0  1
       0 68  9
       1  6 30
[1] "Train: 0.929203539823009"
[1] "Test:  0.0707964601769911"
```

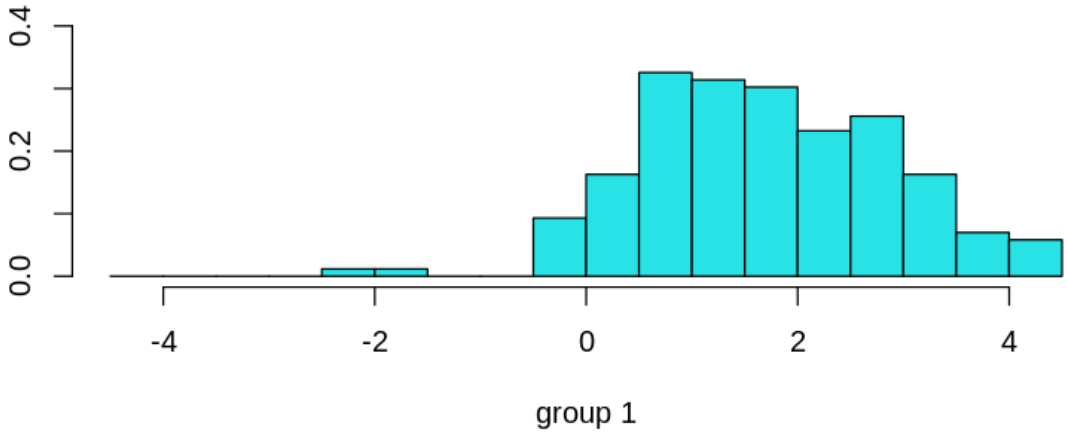
```
[1] "Dim 3 :  1"

glm.pred  0  1
       0 73  0
       1  3 38
[1] "Train: 0.893387314439946"
[1] "Test:  0.106612685560054"
[1] "Dim 3 :  2"

glm.pred  0  1
       0 63  1
       1  1 49
[1] "Train: 0.921727395411606"
[1] "Test:  0.0782726045883941"
[1] "Dim 3 :  3"

glm.pred  0  1
       0 69  3
       1  3 39
[1] "Train: 0.923076923076923"
[1] "Test:  0.0769230769230769"
[1] "Dim 3 :  4"

glm.pred  0  1
       0 71  1
       1  0 41
[1] "Train: 0.921034717494895"
[1] "Test:  0.0789652825051055"
[1] "Dim 3 :  5"

glm.pred  0  1
       0 74  3
       1  0 36
[1] "Train: 0.921034717494895"
[1] "Test:  0.0789652825051055"
```

```r
#2) Using the same data from part (1), apply the "Linear Discriminate" method.
#Include the class predictions for your trained model,
#then plot the results for the  and compare the results.
#-- > Please provide your code, relevant summary/metrics and plots for your model using plot(lda.fit)
#[as given in the lab] #or pairs.lda (MASS).
```

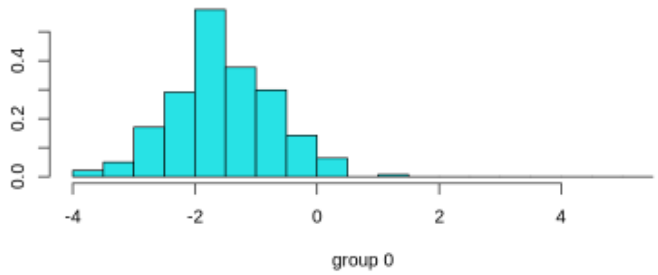
```r
library(MASS)

dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

for (df_name in names(dataframe_list)) {
  current_df <- dataframe_list[[df_name]]

  for (cur_group in 1:5) {
  trainData <- current_df %>%
    filter(data_group != cur_group)
  testData <- current_df %>%
    filter(data_group == cur_group)

  lda.fit <- lda(Diagnosis ~ radius + texture + perimeter + area + smoothness + compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)
  print(paste(df_name, ", Fold",cur_group))
  print(lda.fit)
  plot(lda.fit)
  print("")

  }
}
```

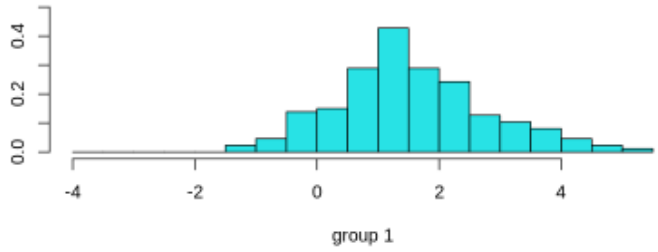
```
[1] "Dim 1 , Fold 1"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimer
    data = trainData)

Prior probabilities of groups:
        0         1
0.6189427 0.3810573

Group means:
    radius  texture perimeter     area smoothness compactness  concavit
0 12.19325 17.97370  78.34915 465.7954 0.09203822  0.07879907 0.0451945
1 17.52393 21.63283 115.81780 983.1642 0.10342665  0.14722179 0.1644464
  concave_points  symmetry fractal_dimension
0     0.02569891 0.1736413        0.06257751
1     0.08926792 0.1933636        0.06287335

Coefficients of linear discriminants:
                             LD1
radius              2.597045485
texture             0.095242575
perimeter          -0.300362618
area               -0.004566093
smoothness          3.070983307
compactness         4.510874526
concavity           2.253055314
concave_points     28.780202724
symmetry            3.951213525
fractal dimension   6.157972309
```


0.0
0.2
0.4
-4
-2
0
2
4
group 0

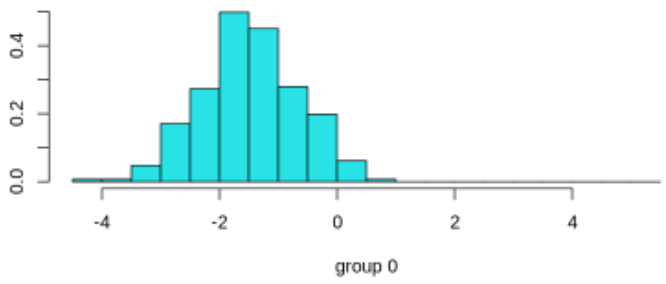

0.0
0.2
0.4
-4
-2
0
2
4
group 1

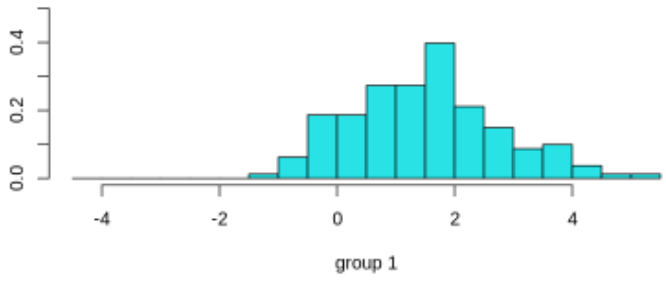
```
[1] "Dim 1 , Fold 2"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6453744 0.3546256

Group means:
    radius  texture perimeter     area smoothness compactness  concavity
0 12.09346 17.73068  77.74539 458.9307 0.09345966  0.08101031 0.04565254
1 17.22087 21.97031 113.59770 952.8876 0.10213335  0.14008323 0.15318994
  concave_points  symmetry fractal_dimension
0     0.02611475 0.1752556        0.06298846
1     0.08430391 0.1904186        0.06234012

Coefficients of linear discriminants:
                        LD1
radius             2.119475876
texture            0.102389395
perimeter         -0.236459993
area              -0.003956042
smoothness        10.154605788
compactness       -2.356536111
concavity          9.069874285
concave_points    19.774710635
symmetry           3.851311320
fractal_dimension  5.323693410
```



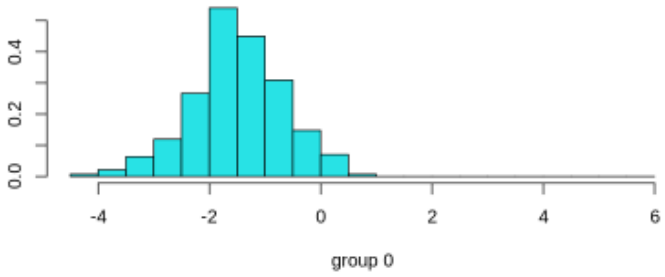


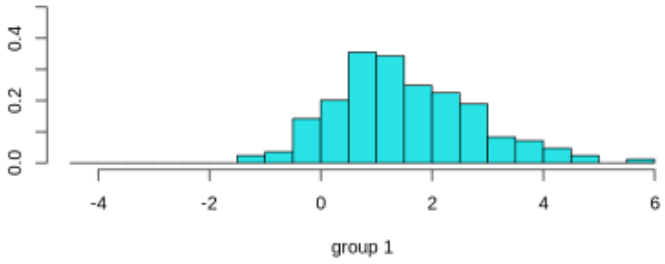
```
[1] "Dim 1 , Fold 3"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6277533 0.3722467

Group means:
    radius  texture perimeter     area smoothness compactness  concavity
0 12.11180 17.89484  77.84151 459.7621 0.09276639  0.08000025 0.04493389
1 17.40799 21.59420 114.88006 973.2189 0.10221544  0.14214929 0.15706112
  concave_points  symmetry fractal_dimension
0     0.02520501 0.1739881        0.06299042
1     0.08628982 0.1924189        0.06225663

Coefficients of linear discriminants:
                              LD1
radius              1.912778172
texture             0.092384903
perimeter          -0.187988909
area               -0.005374465
smoothness         11.925996074
compactness        -1.612395937
concavity           3.913535297
concave_points     28.298999170
symmetry            4.840228278
fractal_dimension  -2.762359571
```



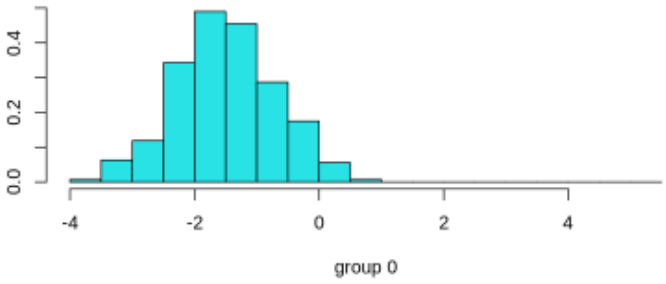


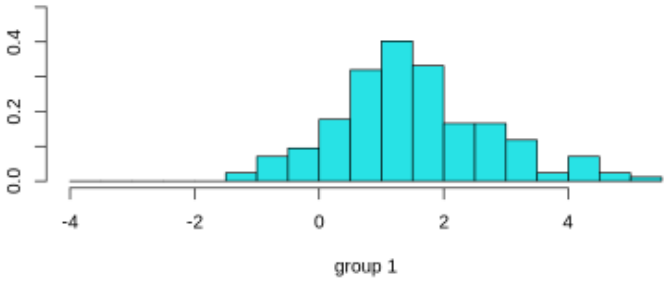
```
[1] "Dim 1 , Fold 4"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6285714 0.3714286

Group means:
    radius  texture perimeter     area smoothness compactness  concavity
0 12.17896 18.08287  78.32829 465.6087 0.09227311  0.08095385 0.04748596
1 17.59503 21.46207 116.34929 993.7414 0.10340414  0.14718006 0.16389527
  concave_points  symmetry fractal_dimension
0     0.02615919 0.1742797        0.06305741
1     0.09005396 0.1942982        0.06277408

Coefficients of linear discriminants:
                              LD1
radius              2.041772399
texture             0.089869260
perimeter          -0.233754582
area               -0.003785609
smoothness          9.935209710
compactness        -1.265734038
concavity           3.284103189
concave_points     31.521577933
symmetry            5.087563569
fractal_dimension  -8.400965917
```



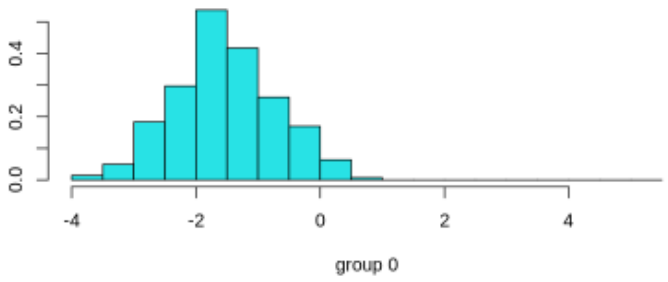


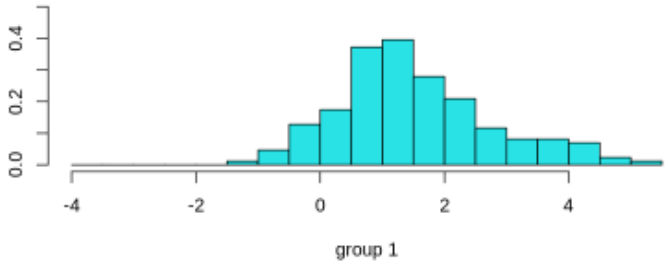
```
[1] "Dim 1 , Fold 5"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
       0        1
0.621978 0.378022

Group means:
    radius  texture perimeter     area smoothness compactness  concavity
0 12.15726 17.89700  78.12527 464.0032 0.09181318  0.07960922 0.04702219
1 17.53959 21.64669 115.90215 986.8640 0.10289727  0.14586866 0.16152390
  concave_points  symmetry fractal_dimension
0     0.02539394 0.1737240        0.06271396
1     0.08842291 0.1927599        0.06275488

Coefficients of linear discriminants:
                            LD1
radius              2.157369751
texture             0.106137315
perimeter          -0.247515922
area               -0.003822039
smoothness          7.257116395
compactness         0.272941483
concavity           0.656687048
concave_points     33.959111528
symmetry            4.881190803
fractal_dimension   7.018372566
```



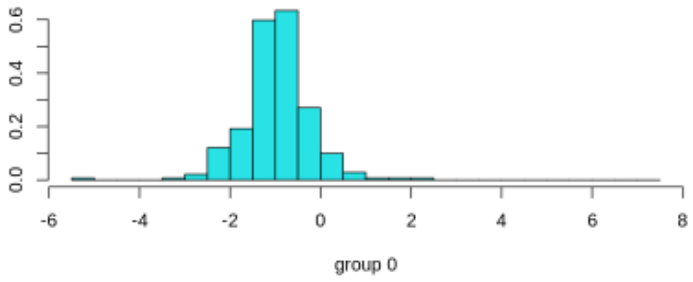


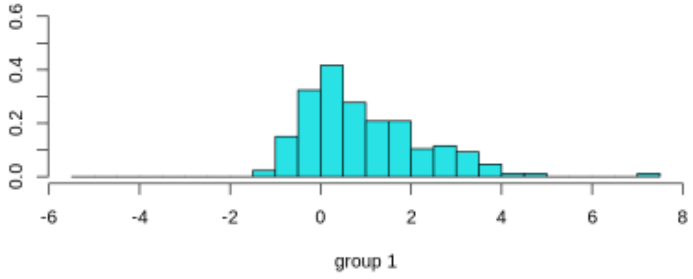
```
[1] "Dim 2 , Fold 1"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6189427 0.3810573

Group means:
     radius  texture perimeter     area  smoothness compactness  concavity
0 0.2831609 1.215372  1.978805 21.20858 0.007135445  0.02072301 0.02560171
1 0.6112081 1.218365  4.336983 72.17133 0.006775543  0.03302651 0.04255353
  concave_points   symmetry fractal_dimension
0    0.009805224 0.02064480       0.003544186
1    0.015118191 0.02045768       0.004165740

Coefficients of linear discriminants:
                              LD1
radius              4.953002e+00
texture            -1.685835e-01
perimeter          -3.950889e-01
area                9.060762e-03
smoothness         -9.244943e+01
compactness         4.493515e+01
concavity          -7.766247e+00
concave_points      8.386156e+01
symmetry           -2.938821e+01
fractal_dimension -2.091753e+02
```



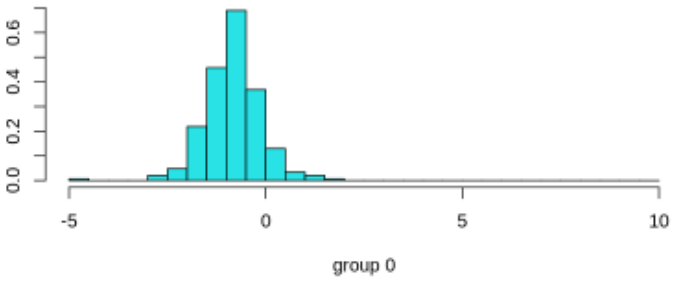


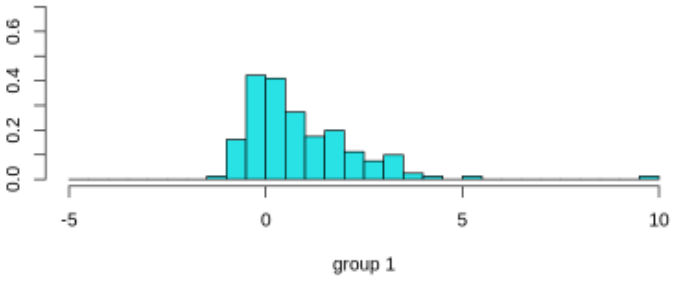
```
[1] "Dim 2 , Fold 2"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6453744 0.3546256

Group means:
     radius  texture perimeter     area  smoothness compactness  concavity
0 0.2883846 1.210295  2.024768 21.36382 0.007394928  0.02156427 0.02532022
1 0.5780478 1.194504  4.079646 67.99422 0.006707366  0.03031973 0.03990839
  concave_points   symmetry fractal_dimension
0    0.009910369 0.02070372       0.003602237
1    0.014619826 0.02003440       0.003845677

Coefficients of linear discriminants:
                              LD1
radius              5.478823e+00
texture            -1.468260e-01
perimeter          -2.340919e-01
area               -2.107893e-05
smoothness         -9.058469e+01
compactness         4.003405e+01
concavity           8.239519e-01
concave_points      4.343702e+01
symmetry           -2.725513e+01
fractal_dimension -2.179745e+02
```



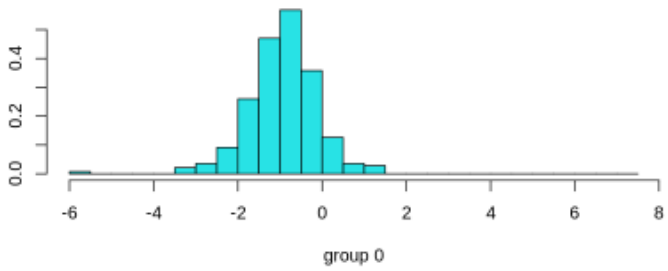


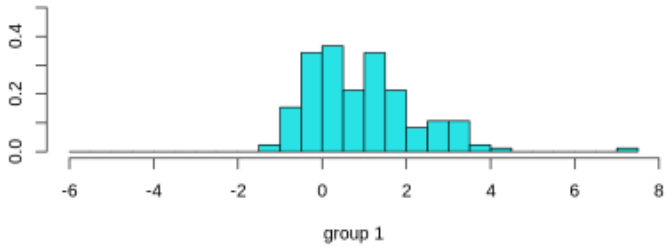
```
[1] "Dim 2 , Fold 3"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6277533 0.3722467

Group means:
     radius  texture perimeter     area  smoothness compactness  concavity
0 0.2845723 1.211872  2.015969 21.09813 0.007220996  0.02152367 0.02542882
1 0.5881201 1.188238  4.163183 70.54343 0.006673124  0.03143983 0.04122503
  concave_points   symmetry fractal_dimension
0    0.009705551 0.02060368       0.003645133
1    0.014761290 0.02002442       0.003972811

Coefficients of linear discriminants:
                              LD1
radius              8.793975e+00
texture            -1.259679e-01
perimeter          -6.335396e-01
area               -7.871333e-03
smoothness         -9.899408e+01
compactness         4.824941e+01
concavity          -4.142997e-01
concave_points      9.869671e+01
symmetry           -3.225411e+01
fractal_dimension -3.266525e+02
```



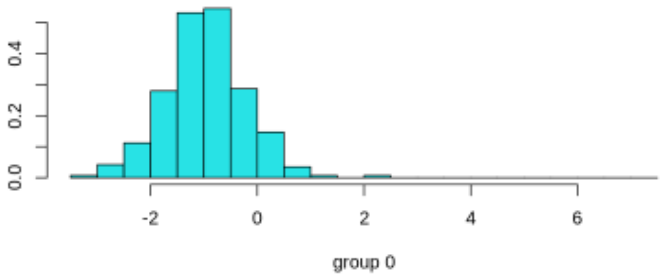


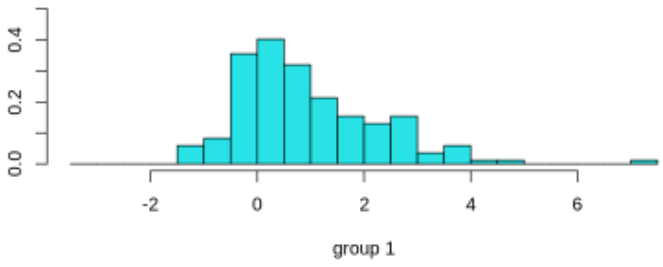
```
[1] "Dim 2 , Fold 4"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6285714 0.3714286

Group means:
     radius  texture perimeter     area  smoothness compactness  concavity
0 0.2838769 1.241231  2.006842 21.19945 0.007157626  0.02204000 0.02689200
1 0.6391645 1.234050  4.536550 76.77533 0.006853426  0.03323334 0.04295432
  concave_points   symmetry fractal_dimension
0     0.01003435 0.02075000       0.003781966
1     0.01545141 0.02093983       0.004141160

Coefficients of linear discriminants:
                              LD1
radius                 7.64442424
texture               -0.20725233
perimeter             -0.40667219
area                  -0.00875696
smoothness           -83.64640719
compactness           43.38344959
concavity             -4.79779250
concave_points        73.73023491
symmetry             -37.79199949
fractal_dimension   -246.92458056
```



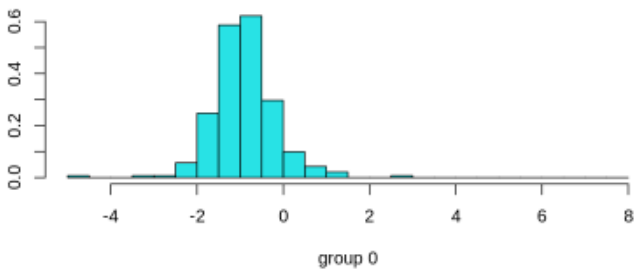


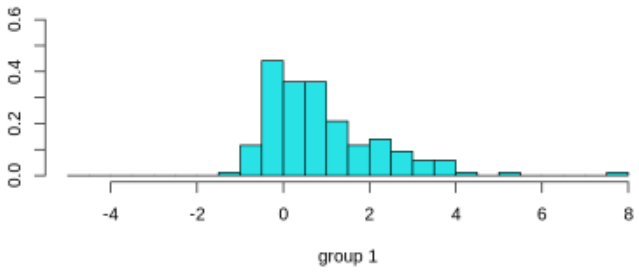
```
[1] "Dim 2 , Fold 5"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
       0        1
0.621978 0.378022

Group means:
     radius  texture perimeter     area  smoothness compactness  concavity
0 0.2802572 1.223290  1.974028 20.79779 0.007063283  0.02132380 0.02675657
1 0.6157337 1.225439  4.389302 73.73849 0.006894663  0.03286883 0.04208442
  concave_points   symmetry fractal_dimension
0    0.009829735 0.02021113       0.003605669
1    0.015305901 0.02065581       0.004122442

Coefficients of linear discriminants:
                              LD1
radius              6.879959e+00
texture            -1.372559e-01
perimeter          -4.446095e-01
area               -3.563124e-03
smoothness         -9.952935e+01
compactness         4.346440e+01
concavity          -9.472997e+00
concave_points      9.167448e+01
symmetry           -2.470254e+01
fractal_dimension -2.003737e+02
```



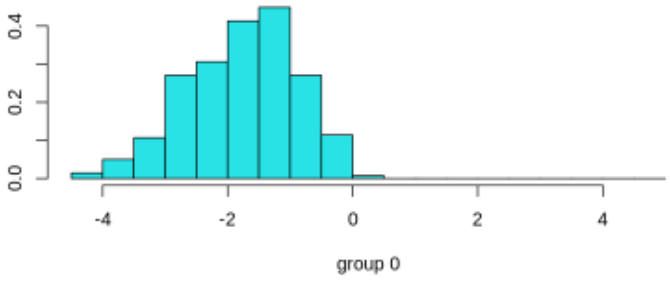


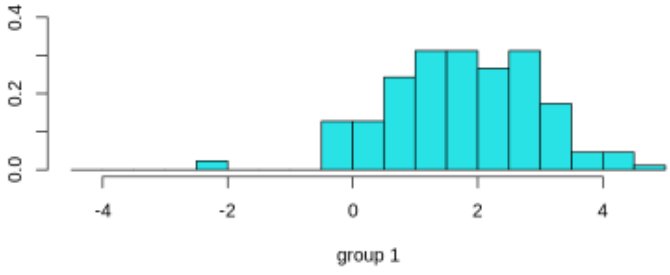
```
[1] "Dim 3 , Fold 1"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6189427 0.3810573

Group means:
    radius  texture perimeter      area smoothness compactness concavity
0 13.43548 23.57096  87.31046  562.9587  0.1245616   0.1788892 0.1636143
1 21.19410 29.32902 141.72809 1424.3133  0.1453456   0.3816423 0.4583086
  concave_points  symmetry fractal_dimension
0     0.07469018 0.2705260        0.07899125
1     0.18339688 0.3241659        0.09234364

Coefficients of linear discriminants:
                            LD1
radius              0.700384451
texture             0.050886020
perimeter          -0.016376825
area               -0.002938251
smoothness          7.495364184
compactness        -1.648481850
concavity           0.588017675
concave_points      8.676006047
symmetry            2.437341984
fractal_dimension 13.238609870
```



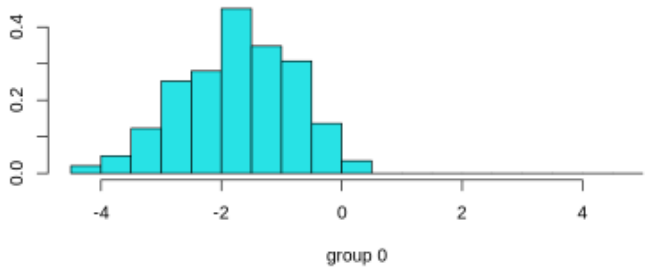


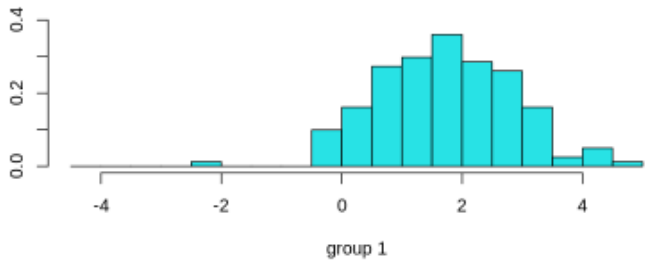
```
[1] "Dim 3 , Fold 2"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6453744 0.3546256

Group means:
    radius  texture perimeter      area smoothness compactness concavity
0 13.32736 23.26580  86.61116  554.5266  0.1263383   0.1829296 0.1633274
1 20.86863 29.87435 139.40752 1394.6298  0.1452539   0.3654576 0.4389750
  concave_points  symmetry fractal_dimension
0     0.07392488 0.2704737        0.07939369
1     0.17882609 0.3230578        0.09076677

Coefficients of linear discriminants:
                        LD1
radius             0.573449427
texture            0.059005755
perimeter          0.001227693
area              -0.002903105
smoothness        10.729389896
compactness       -3.037577462
concavity          1.839982303
concave_points     6.612873163
symmetry           2.987065598
fractal_dimension 11.867746423
```



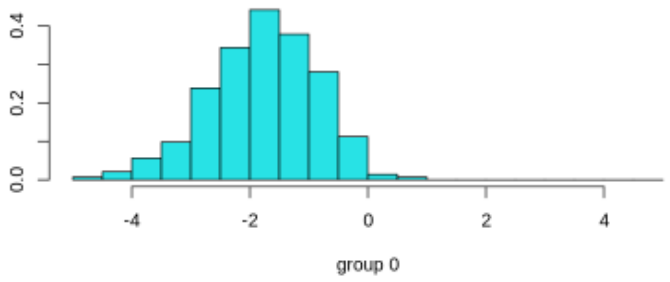


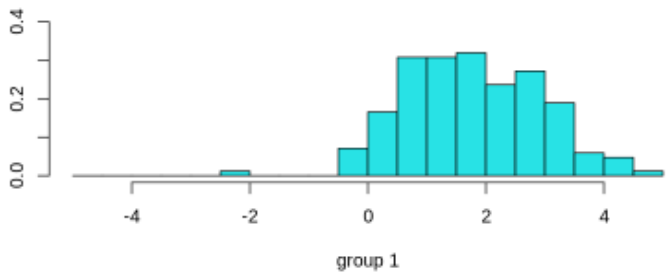
```
[1] "Dim 3 , Fold 3"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6277533 0.3722467

Group means:
    radius  texture perimeter      area smoothness compactness concavity
0 13.34758 23.47751  86.83688  555.6133  0.1254921   0.1831679 0.1637454
1 20.98680 29.32633 140.09822 1403.4432  0.1444776   0.3701864 0.4511876
  concave_points  symmetry fractal_dimension
0     0.07387481 0.2695954        0.07943530
1     0.18127006 0.3220923        0.09127929

Coefficients of linear discriminants:
                             LD1
radius              0.5539583401
texture             0.0534507003
perimeter           0.0008032184
area               -0.0028080165
smoothness         11.4725960123
compactness        -3.2615140686
concavity           1.7106008284
concave_points      8.7675845494
symmetry            4.3836740654
fractal_dimension   7.8529609679
```



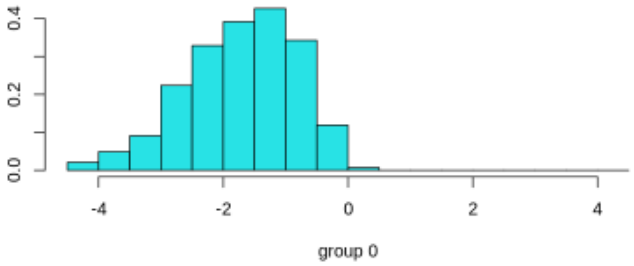


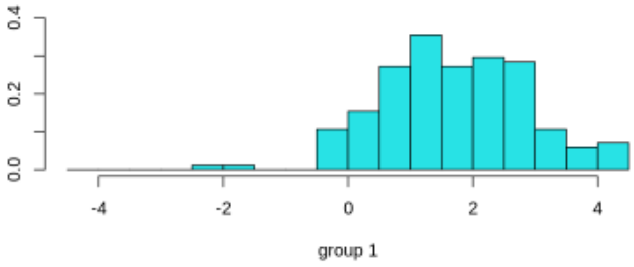
```
[1] "Dim 3 , Fold 4"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
        0         1
0.6285714 0.3714286

Group means:
    radius  texture perimeter     area smoothness compactness concavity
0 13.40424 23.76535  87.25633  561.650  0.1242022   0.1851222 0.1696276
1 21.32296 29.09006 142.78757 1446.621  0.1443846   0.3729802 0.4510472
  concave_points  symmetry fractal_dimension
0      0.0752420 0.2713024        0.08000448
1      0.1829704 0.3231136        0.09105533

Coefficients of linear discriminants:
                              LD1
radius              0.607861451
texture             0.051361563
perimeter          -0.006578893
area               -0.002672586
smoothness         11.623851430
compactness        -3.341355408
concavity           1.208322966
concave_points      7.737620250
symmetry            2.690168792
fractal_dimension  12.901343568
```





```
[1] "Dim 3 , Fold 5"
Call:
lda(Diagnosis ~ radius + texture + perimeter + area + smoothness +
    compactness + concavity + concave_points + symmetry + fractal_dimension,
    data = trainData)

Prior probabilities of groups:
       0        1
0.621978 0.378022

Group means:
    radius  texture perimeter      area smoothness compactness concavity
0 13.38655 23.50254  87.02951  559.9258  0.1241559   0.1831886 0.1709399
1 21.18616 29.28174 141.69983 1426.8628  0.1443697   0.3763402 0.4466621
  concave_points  symmetry fractal_dimension
0     0.07450551 0.2693194        0.07937827
1     0.18256017 0.3216721        0.09150203

Coefficients of linear discriminants:
                              LD1
radius              0.516577353
texture             0.058673963
perimeter           0.007626326
area               -0.002694230
smoothness         10.037060037
compactness        -3.076857855
concavity           0.757905949
concave_points      7.867249519
symmetry            2.384675470
fractal_dimension  15.272867381
```





```
#(3) Implement and assess a Classification model:
# A) Select a dataset and classification TARGET and
#select the relevant features you wish to use.
#[You may use data from ISLR or MASS -- or one of the sets we've worked with so far].
#You can either choose a binary classification (where applicable) or
#a multi-class target using the LDA method.
#You may use either a single factor classification (only one feature) or
#multiple features (I suggest you limit your selection to 5).
#-- No performance metrics are required in this case,
#we will assess feature selection in a later assignment.
#Select the division scale (test set size) and
#perform a fit for your model on the training data.
#B ) Repeat the process from the previous step using the k-fold methods:
#
# Recall selection of k-fold number is important,
#you may select any valid value for the range we've discussed.
```

```
library(class)

#Import WDBC Data
#(1) Use the dataset provided above
source_file_url = "https://raw.githubusercontent.com/joelvinas/COMP-SCI_5565/refs/heads/main/Assignment%203/Data/wdbc.data"

wdbc_data <- read.csv(source_file_url)
all_headers <- c("ID", "Diagnosis",
                 "radius1", "texture1", "perimeter1", "area1", "smoothness1", "compactness1", "concavity1", "concave_points1", "symmetry1", "fractal_dimension1",
                 "radius2", "texture2", "perimeter2", "area2", "smoothness2", "compactness2", "concavity2", "concave_points2", "symmetry2", "fractal_dimension2",
                 "radius3", "texture3", "perimeter3", "area3", "smoothness3", "compactness3", "concavity3", "concave_points3", "symmetry3", "fractal_dimension3")

colnames(wdbc_data) <- all_headers
#wdbc_data$Diagnosis[wdbc_data$Diagnosis == "M"] <- 1   #M for Malignant
#wdbc_data$Diagnosis[wdbc_data$Diagnosis == "B"] <- 0  #B for Benign

# Gemini Code suggestion: Robustly convert Diagnosis to numeric 0/1 (0 for 'B', 1 for 'M')
wdbc_data$Diagnosis <- as.numeric(wdbc_data$Diagnosis == "M")

#Break the dataset into 5 parts for later 5-Fold analysis
num_parts <- 5
wdbc_data$data_group <- rep(1:num_parts, length.out = nrow(wdbc_data))

#Split the dataset into the 3 dimensions
wdbc_data1 <- wdbc_data[, c(1,2, 3:12, 33)]
wdbc_data2 <- wdbc_data[, c(1,2, 13:22, 33)]
wdbc_data3 <- wdbc_data[, c(1,2, 23:32, 33)]

# Define new headers for subset
part_headers <- c("ID", "Diagnosis", "radius", "texture", "perimeter", "area", "smoothness", "compactness", "concavity", "concave_points", "symmetry", "fractal_dimension", "data_group")

# Assign the new headers to the new data frames
colnames(wdbc_data1) <- part_headers
colnames(wdbc_data2) <- part_headers
colnames(wdbc_data3) <- part_headers

# This loop was already present in the notebook; leaving it as is for now.
# It prints head and dim information for debugging purposes but doesn't modify dataframes.
dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

results_df <- data.frame(
  name = character(0),
  accuracy = numeric(0),
  datasource = character(0),
  datagroup = integer(0),
  stringsAsFactors = FALSE
)
```

```r
for (df_name in names(dataframe_list)) {
  current_df <- dataframe_list[[df_name]]

    # Select up to 5 features
    # Features were selected due to their statistical relation to the model in previous analysis
    # It would have been nice to have this selected programatically, and on a per Dimension basis!
  k_features <- current_df[, c("radius", "texture", "perimeter", "area", "smoothness")]

  # Scale the selected features
  k_scaled_features <- scale(k_features)

  # Extract the Diagnosis column as the target variable
  k_diagnosis <- current_df$Diagnosis

  k_fold_accuracy <- c()
  k_value <- 5 # Define k once

  for (cur_group in 1:5) {
    # Split data into training and testing sets based on DataGroup

    train_indices <- which(current_df$data_group != cur_group)
    test_indices <- which(current_df$data_group == cur_group)

    print(paste(df_name, cur_group))
    #print(head(current_df))

    train_features <- k_scaled_features[train_indices, , drop = FALSE]
    test_features <- k_scaled_features[test_indices, , drop = FALSE]

    print(paste("Number of rows in k_scaled_features:", nrow(k_scaled_features)))
    print(paste("Number of rows in train_features:", nrow(train_features)))
    print(paste("Number of rows in test_features:", nrow(test_features)))

    # Check if training set is empty or has zero rows
    if (nrow(train_features) == 0) {
      cat(paste0("Reason: Training set for Fold ", cur_group, " is empty. This indicates a problem with data splitting or original data availability. Skipping this fold.\n"))
      next
    }

    # Convert diagnosis to factor with explicit levels (0 and 1) to avoid issues with missing levels
    train_diagnosis_factor <- factor(k_diagnosis[train_indices], levels = c(0, 1))
    test_diagnosis_factor <- factor(k_diagnosis[test_indices], levels = c(0, 1))

    cat(paste0("\n--- Fold ", cur_group, " Training Class Distribution ---\n"))
    diagnosis_counts <- table(train_diagnosis_factor)
    print(diagnosis_counts)

    # Check if all classes are present in the training set
    if (length(diagnosis_counts) < 2) {
        cat(paste0("Reason: Training set for Fold ", cur_group, " does not contain both classes (0 and 1). Skipping this fold.\n"))
        next
    }

    # Check if all classes have at least k_value samples in the training set
    if (any(diagnosis_counts < k_value)) {
      cat(paste0("Reason: Insufficient training samples for one or more classes to run KNN with k=", k_value, ". Skipping this fold.\n"))
      next # Skip this fold if conditions are not met
    }

    # Train KNN model using the k-value (k=5)
    knn_pred <- knn(
      train = train_features,
      test = test_features,
      cl = train_diagnosis_factor, # Use the factor version of training labels
      k = k_value
    )

    # Ensure predictions are also factors with correct levels for confusion matrix
    knn_pred_factor <- factor(knn_pred, levels = c(0, 1))

    # Print Confusion Matrix for current fold
    cat(paste0("\n--- Fold ", cur_group, " Results ---\n"))
    confusion_matrix <- table(knn_pred_factor, test_diagnosis_factor)
    print(confusion_matrix)

    # Calculate and print Accuracy for current fold
    accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
    cat(paste0("Accuracy: ", round(accuracy, 4), "\n"))
    k_fold_accuracy <- c(k_fold_accuracy, accuracy)
  }

  if (length(k_fold_accuracy) > 0) {
      cat("\nAverage K-Fold Accuracy for ", df_name, ": ", mean(k_fold_accuracy), "\n")
  } else {
      cat("\nNo folds were processed successfully for ", df_name, " due to insufficient training data for KNN.\n")
  }

  # Add new row to results
  new_results <- data.frame(
    name = paste(df_name,cur_group),
    accuracy = k_fold_accuracy,
    datasource = df_name,
    datagroup = cur_group,
    stringsAsFactors = FALSE
  )

  # Add the new row
  results_df <- rbind(results_df, new_results)

}
```

```
[1] "Dim 1 1"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 1 Training Class Distribution ---
train_diagnosis_factor
  0   1
281 173

--- Fold 1 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 73  7
              1  3 31
Accuracy: 0.9123
  [1] "Dim 1 2"
  [1] "Number of rows in k_scaled_features: 568"
  [1] "Number of rows in train_features: 454"
  [1] "Number of rows in test_features: 114"

  --- Fold 2 Training Class Distribution ---
  train_diagnosis_factor
    0   1
  293 161

  --- Fold 2 Results ---
                 test_diagnosis_factor
  knn_pred_factor  0  1
                0 62  6
                1  2 44
  Accuracy: 0.9298
```

```
[1] "Dim 1 3"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 3 Training Class Distribution ---
train_diagnosis_factor
  0   1
285 169

--- Fold 3 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 66  3
              1  6 39
Accuracy: 0.9211
[1] "Dim 1 4"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 4 Training Class Distribution ---
train_diagnosis_factor
  0   1
286 169

--- Fold 4 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 69  5
              1  2 37
Accuracy: 0.9381
```

```
[1] "Dim 1 5"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 5 Training Class Distribution ---
train_diagnosis_factor
  0   1
283 172

--- Fold 5 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 69  9
              1  5 30
Accuracy: 0.8761
Average K-Fold Accuracy for  Dim 1 :  0.9154634
[1] "Dim 2 1"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 1 Training Class Distribution ---
train_diagnosis_factor
  0   1
281 173

--- Fold 1 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 67 12
              1  9 26
Accuracy: 0.8158
```

```
[1] "Dim 2 2"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 2 Training Class Distribution ---
train_diagnosis_factor
  0   1
293 161

--- Fold 2 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 60  6
              1  4 44
Accuracy: 0.9123
```

```
[1] "Dim 2 3"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 3 Training Class Distribution ---
train_diagnosis_factor
  0   1
285 169

--- Fold 3 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 64  9
              1  8 33
Accuracy: 0.8509
```

```
[1] "Dim 2 4"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 4 Training Class Distribution ---
train_diagnosis_factor
  0   1
286 169

--- Fold 4 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 66 20
              1  5 22
Accuracy: 0.7788
```

```
[1] "Dim 2 5"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 5 Training Class Distribution ---
train_diagnosis_factor
  0   1
283 172

--- Fold 5 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 67 13
              1  7 26
Accuracy: 0.823
```

```
Average K-Fold Accuracy for  Dim 2 :  0.8361435
```

```
[1] "Dim 3 1"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 1 Training Class Distribution ---
train_diagnosis_factor
  0   1
281 173

--- Fold 1 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 73  2
              1  3 36
Accuracy: 0.9561
```

```
[1] "Dim 3 2"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 2 Training Class Distribution ---
train_diagnosis_factor
  0   1
293 161

--- Fold 2 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 63  2
              1  1 48
Accuracy: 0.9737
```

```
[1] "Dim 3 3"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 454"
[1] "Number of rows in test_features: 114"

--- Fold 3 Training Class Distribution ---
train_diagnosis_factor
  0   1
285 169

--- Fold 3 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 71  2
              1  1 40
Accuracy: 0.9737
```

```
[1] "Dim 3 4"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 4 Training Class Distribution ---
train_diagnosis_factor
  0   1
286 169

--- Fold 4 Results ---
               test_diagnosis_factor
knn_pred_factor  0  1
              0 71  0
              1  0 42
Accuracy: 1
```

```
[1] "Dim 3 5"
[1] "Number of rows in k_scaled_features: 568"
[1] "Number of rows in train_features: 455"
[1] "Number of rows in test_features: 113"

--- Fold 5 Training Class Distribution ---
train_diagnosis_factor
  0   1
283 172

--- Fold 5 Results ---
                test_diagnosis_factor
knn_pred_factor  0  1
              0 73  4
              1  1 35
Accuracy: 0.9558
```

```
Average K-Fold Accuracy for  Dim 3 :  0.9718522
```

```r
library(dplyr)

dataframe_list <- list("Dim 1" = wdbc_data1, "Dim 2" = wdbc_data2, "Dim 3" = wdbc_data3)

for (df_name in names(dataframe_list)) {

mean_accuracy <- results_df %>%
  filter(datasource == df_name) %>%
  summarize(mean_acc = mean(accuracy))

print(paste("Mean Accuracy for ",df_name,":", mean_accuracy$mean_acc))

}
```

```
[1] "Mean Accuracy for  Dim 1 : 0.915463437354448"
[1] "Mean Accuracy for  Dim 2 : 0.836143455985095"
[1] "Mean Accuracy for  Dim 3 : 0.971852196863841"
```